DATA 260 Homework 8

Xuanhua Li
019122974

Model Name: MacBook Air

Model Identifier: Mac16,13

Chip: Apple M4

Memory: 16 GB

Ollama qwen3:8b

Implementation and Evidence Commit: 60c16977c37df9a07ee65166388eb4185dbb1a91

GitHub Repository: https://github.com/carrovo/data260-2974

Submission Branch: hw8-work

Submission Tag: hw8

Configuration	Value
SID4	2974
PORT_BASE	8274
PREFIX	s2974
SEED	2974
VERIFY_SEED	262974
DOMAIN_ID	6
Primary domain entity	Rental listing

Part 1 – Kafka Request/Response Service

1.1 Architecture

The request/response service contains four main components:

1. A client sends a rental-listing operation to FastAPI.
2. FastAPI publishes the request to Kafka.
3. A Worker consumes, validates, and processes the request.
4. The Worker publishes a response that FastAPI returns to the client.

The entities are stored in a Python dictionary inside the Worker. Each successfully created listing receives a UUID. The service validates required fields, rejects unknown operations, and returns clean error responses containing the original correlation_id.

1.2 Kafka Topics

Exactly two prefixed topics are used:

Topic	Purpose	Partitions
s2974_entity_requests	Carries CREATE and GET requests from FastAPI to the Worker	1
s2974_entity_responses	Carries Worker responses back to FastAPI	1

The FastAPI application runs at: http://127.0.0.1:8274

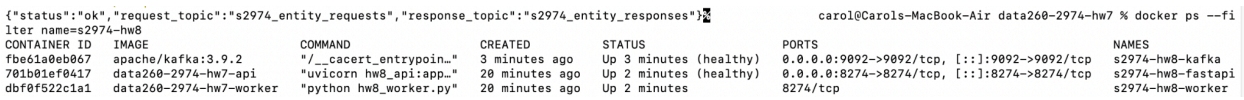


Figure 1. Kafka, Worker, and FastAPI containers running

1.3 Request and Response Schemas

Every request uses the following envelope:

```
{
  "correlation_id": "unique-id",
  "reply_to": "s2974_entity_responses",
  "operation": "CREATE_LISTING or GET_LISTING",
  "data": {}
}
```

A successful CREATE response has this form:


```
{
  "correlation_id": "same-correlation-id",
  "success": true,
  "entity_id": "generated-uuid",
  "message": "Rental listing created"
}
```


A successful GET response has this form:


```
{
  "correlation_id": "same-correlation-id",
  "success": true,
  "entity": {
    "entity_id": "existing-entity-id",
    "listingTitle": "...",
    "address": "...",
    "description": "...",
    "propertyType": "...",
    "monthlyRent": 1800,
    "availableUnits": 1
  }
}
```


Error responses contain success: false, a clear error message, and the unchanged correlation_id.

1.4 CREATE_LISTING and GET_LISTING Demonstration

The demonstration first sent a CREATE_LISTING request for a studio near SJSU. The Worker generated: entity_id = 5fcda188-adaa-4eb4-984c-15afddce2d3d

The CREATE request and response preserved: correlation_id = 5f835fe5-0e35-46a0-96f1-81482b097973

```
carol@Carols-MacBook-Air data260-2974-hw7 % PYTHONPATH=code .venv/bin/python code/homework_demo.py

[CLIENT] CREATE_LISTING REQUEST
{
  "correlation_id": "5f835fe5-0e35-46a0-96f1-81482b097973",
  "data": {
    "address": "100 Example Street, San Jose, CA",
    "availableUnits": 1,
    "description": "A furnished studio located close to the SJSU campus.",
    "listingTitle": "Studio Near SJSU",
    "monthlyRent": 1800,
    "propertyType": "Studio"
  },
  "operation": "CREATE_LISTING",
  "reply_to": "s2974_entity_responses"
}

[CLIENT] CREATE_LISTING RESPONSE
{
  "correlation_id": "5f835fe5-0e35-46a0-96f1-81482b097973",
  "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
  "message": "Rental listing created",
  "success": true
}
[CLIENT] Extracted entity_id=5fcda188-adaa-4eb4-984c-15afddce2d3d
```

Figure 2. CREATE_LISTING request and successful response

The client then sent GET_LISTING using the generated entity ID. The response returned the same listing data and preserved the GET request's correlation ID:

correlation_id = de9a2288-50e0-4985-9bbb-75b219ce11a9

```
[CLIENT] GET_LISTING REQUEST
{
  "correlation_id": "de9a2288-50e0-4985-9bbb-75b219ce11a9",
  "data": {
    "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d"
  },
  "operation": "GET_LISTING",
  "reply_to": "s2974_entity_responses"
}

[CLIENT] GET_LISTING RESPONSE
{
  "correlation_id": "de9a2288-50e0-4985-9bbb-75b219ce11a9",
  "entity": {
    "address": "100 Example Street, San Jose, CA",
    "availableUnits": 1,
    "description": "A furnished studio located close to the SJSU campus.",
    "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
    "listingTitle": "Studio Near SJSU",
    "monthlyRent": 1800,
    "propertyType": "Studio"
  },
  "success": true
}

[CLIENT] RETRIEVED RENTAL LISTING
{
  "address": "100 Example Street, San Jose, CA",
  "availableUnits": 1,
  "description": "A furnished studio located close to the SJSU campus.",
  "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
  "listingTitle": "Studio Near SJSU",
  "monthlyRent": 1800,
  "propertyType": "Studio"
}

[CLIENT] PASS: CREATE_LISTING -> GET_LISTING completed
[CLIENT] PASS: each correlation_id was preserved end to end
```

Figure 3. GET_LISTING request and retrieved entity

1.5 Complete Kafka Message Flow

FastAPI logged the request before publishing it to s2974_entity_requests. It then logged the Kafka partition and offset, received the matching response from s2974_entity_responses, and returned the HTTP response.

```
s2974-hw8-fastapi  | [FASTAPI] SENDING REQUEST topic=s2974_entity_requests correlation_id=5f835fe5-0e35-46a0-96f1-81482b097973
s2974-hw8-fastapi  | [FASTAPI] REQUEST {
s2974-hw8-fastapi  |   "correlation_id": "5f835fe5-0e35-46a0-96f1-81482b097973",
s2974-hw8-fastapi  |   "data": {
s2974-hw8-fastapi  |     "address": "100 Example Street, San Jose, CA",
s2974-hw8-fastapi  |     "availableUnits": 1,
s2974-hw8-fastapi  |     "description": "A furnished studio located close to the SJSU campus.",
s2974-hw8-fastapi  |     "listingTitle": "Studio Near SJSU",
s2974-hw8-fastapi  |     "monthlyRent": 1800,
s2974-hw8-fastapi  |     "propertyType": "Studio"
s2974-hw8-fastapi  |   },
s2974-hw8-fastapi  |   "operation": "CREATE_LISTING",
s2974-hw8-fastapi  |   "reply_to": "s2974_entity_responses"
s2974-hw8-fastapi  | }
s2974-hw8-fastapi  | [FASTAPI] SENT REQUEST topic=s2974_entity_requests partition=0 offset=0
s2974-hw8-fastapi  | [FASTAPI] RECEIVED RESPONSE topic=s2974_entity_responses partition=0 offset=0 correlation_id=5f835fe5-0e35-46a0-96f1-81482b097973
s2974-hw8-fastapi  | [FASTAPI] RESPONSE {
s2974-hw8-fastapi  |   "correlation_id": "5f835fe5-0e35-46a0-96f1-81482b097973",
s2974-hw8-fastapi  |   "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
s2974-hw8-fastapi  |   "message": "Rental listing created",
s2974-hw8-fastapi  |   "success": true
s2974-hw8-fastapi  | }
s2974-hw8-fastapi  | [FASTAPI] RETURNING HTTP RESPONSE correlation_id=5f835fe5-0e35-46a0-96f1-81482b097973
s2974-hw8-fastapi  | INFO:     192.168.65.1:57021 - "POST /hw8/listings HTTP/1.1" 200 OK
s2974-hw8-fastapi  | [FASTAPI] SENDING REQUEST topic=s2974_entity_requests correlation_id=de9a2288-50e0-4985-9bbb-75b219ce11a9
s2974-hw8-fastapi  | [FASTAPI] REQUEST {
s2974-hw8-fastapi  |   "correlation_id": "de9a2288-50e0-4985-9bbb-75b219ce11a9",
s2974-hw8-fastapi  |   "data": {
s2974-hw8-fastapi  |     "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d"
s2974-hw8-fastapi  |   },
s2974-hw8-fastapi  |   "operation": "GET_LISTING",
s2974-hw8-fastapi  |   "reply_to": "s2974_entity_responses"
s2974-hw8-fastapi  | }
s2974-hw8-fastapi  | [FASTAPI] SENT REQUEST topic=s2974_entity_requests partition=0 offset=1
s2974-hw8-fastapi  | [FASTAPI] RECEIVED RESPONSE topic=s2974_entity_responses partition=0 offset=1 correlation_id=de9a2288-50e0-4985-9bbb-75b219ce11a9
s2974-hw8-fastapi  | [FASTAPI] RESPONSE {
s2974-hw8-fastapi  |   "correlation_id": "de9a2288-50e0-4985-9bbb-75b219ce11a9",
s2974-hw8-fastapi  |   "entity": {
s2974-hw8-fastapi  |     "address": "100 Example Street, San Jose, CA",
s2974-hw8-fastapi  |     "availableUnits": 1,
s2974-hw8-fastapi  |     "description": "A furnished studio located close to the SJSU campus.",
s2974-hw8-fastapi  |     "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
s2974-hw8-fastapi  |     "listingTitle": "Studio Near SJSU",
s2974-hw8-fastapi  |     "monthlyRent": 1800,
s2974-hw8-fastapi  |     "propertyType": "Studio"
s2974-hw8-fastapi  |   },
s2974-hw8-fastapi  |   "success": true
s2974-hw8-fastapi  | }
s2974-hw8-fastapi  | [FASTAPI] RETURNING HTTP RESPONSE correlation_id=de9a2288-50e0-4985-9bbb-75b219ce11a9
s2974-hw8-fastapi  | INFO:     192.168.65.1:52336 - "POST /hw8/listings HTTP/1.1" 200 OK
s2974-hw8-fastapi  | INFO:     127.0.0.1:41034 - "GET /health HTTP/1.1" 200 OK
s2974-hw8-fastapi  | INFO:     127.0.0.1:35236 - "GET /health HTTP/1.1" 200 OK
```

Figure 4. FastAPI request/response Kafka flow

The Worker logged each step in order:

RECEIVED → REQUEST → PROCESSED → RESPONSE → SENT → COMMITTED

The CREATE request used request offset 0, and the GET request used request offset 1. The Worker committed each request only after publishing its response.

```
s2974-hw8-worker   | Group Coordinator Request failed: [Error 15] GroupCoordinatorNotAvailableError
s2974-hw8-worker   | Marking the coordinator dead (node 1)for group s2974_listing_worker.
s2974-hw8-worker   | Marking the coordinator dead (node 1)for group s2974_listing_worker.
s2974-hw8-worker   | Consumer is ready and listening... topic=s2974_entity_requests
s2974-hw8-worker   | [WORKER] RECEIVED topic=s2974_entity_requests partition=0 offset=0
s2974-hw8-worker   | [WORKER] REQUEST {
s2974-hw8-worker   |   "correlation_id": "5f835fe5-0e35-46a0-96f1-81482b097973",
s2974-hw8-worker   |   "data": {
s2974-hw8-worker   |     "address": "100 Example Street, San Jose, CA",
s2974-hw8-worker   |     "availableUnits": 1,
s2974-hw8-worker   |     "description": "A furnished studio located close to the SJSU campus.",
s2974-hw8-worker   |     "listingTitle": "Studio Near SJSU",
s2974-hw8-worker   |     "monthlyRent": 1800,
s2974-hw8-worker   |     "propertyType": "Studio"
s2974-hw8-worker   |   },
s2974-hw8-worker   |   "operation": "CREATE_LISTING",
s2974-hw8-worker   |   "reply_to": "s2974_entity_responses"
s2974-hw8-worker   | }
s2974-hw8-worker   | [WORKER] PROCESSED operation=CREATE_LISTING correlation_id=5f835fe5-0e35-46a0-96f1-81482b097973
s2974-hw8-worker   | [WORKER] RESPONSE {
s2974-hw8-worker   |   "correlation_id": "5f835fe5-0e35-46a0-96f1-81482b097973",
s2974-hw8-worker   |   "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
s2974-hw8-worker   |   "message": "Rental listing created",
s2974-hw8-worker   |   "success": true
s2974-hw8-worker   | }
s2974-hw8-worker   | [WORKER] SENT topic=s2974_entity_responses partition=0 offset=0
s2974-hw8-worker   | [WORKER] COMMITTED request_offset=0
s2974-hw8-worker   | [WORKER] RECEIVED topic=s2974_entity_requests partition=0 offset=1
s2974-hw8-worker   | [WORKER] REQUEST {
s2974-hw8-worker   |   "correlation_id": "de9a2288-50e0-4985-9bbb-75b219ce11a9",
s2974-hw8-worker   |   "data": {
s2974-hw8-worker   |     "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d"
s2974-hw8-worker   |   },
s2974-hw8-worker   |   "operation": "GET_LISTING",
s2974-hw8-worker   |   "reply_to": "s2974_entity_responses"
s2974-hw8-worker   | }
s2974-hw8-worker   | [WORKER] PROCESSED operation=GET_LISTING correlation_id=de9a2288-50e0-4985-9bbb-75b219ce11a9
s2974-hw8-worker   | [WORKER] RESPONSE {
s2974-hw8-worker   |   "correlation_id": "de9a2288-50e0-4985-9bbb-75b219ce11a9",
s2974-hw8-worker   |   "entity": {
s2974-hw8-worker   |     "address": "100 Example Street, San Jose, CA",
s2974-hw8-worker   |     "availableUnits": 1,
s2974-hw8-worker   |     "description": "A furnished studio located close to the SJSU campus.",
s2974-hw8-worker   |     "entity_id": "5fcda188-adaa-4eb4-984c-15afddce2d3d",
s2974-hw8-worker   |     "listingTitle": "Studio Near SJSU",
s2974-hw8-worker   |     "monthlyRent": 1800,
s2974-hw8-worker   |     "propertyType": "Studio"
s2974-hw8-worker   |   },
s2974-hw8-worker   |   "success": true
s2974-hw8-worker   | }
s2974-hw8-worker   | [WORKER] SENT topic=s2974_entity_responses partition=0 offset=1
s2974-hw8-worker   | [WORKER] COMMITTED request_offset=1
```

Figure 5. Worker processing, response, and offset commits

The complete demonstration passed both required checks:

CREATE_LISTING → GET_LISTING completed
Each correlation_id was preserved end to end

Part 2 – Three-Agent Kafka Pipeline

2.1 Workflow

Planner, Writer, and Reviewer run as independent processes. No agent directly invokes another agent. Kafka topics connect the stages.

Stage	Reads from	Publishes to
Client	—	s2974_inbox
Planner	s2974_inbox	s2974_tasks
Writer	s2974_tasks	s2974_drafts
Reviewer	s2974_drafts	s2974_final

The topics were configured as follows:

Topic	Partitions
s2974_inbox	1
s2974_tasks	2
s2974_drafts	1
s2974_final	1

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml ps -a \
  planner writer reviewer
NAME                 IMAGE                       COMMAND                  SERVICE    CREATED             STATUS             PORTS
s2974-hw8-planner    data260-2974-hw7-planner    "python planner.py"      planner    About an hour ago   Up About an hour   8274/tcp
s2974-hw8-reviewer   data260-2974-hw7-reviewer   "python reviewer.py"     reviewer   About an hour ago   Up About an hour   8274/tcp
s2974-hw8-writer     data260-2974-hw7-writer     "python writer.py"       writer     About an hour ago   Up About an hour   8274/tcp
```

Figure 6. Planner, Writer, Reviewer, Kafka, and supporting containers

2.2 End-to-End Message Journey

The client submitted this question: What information should a renter verify before applying for a rental listing?

The client created this correlation ID once: 1524d90b-c744-4303-995f-ed4d9c2d43de

The same value was preserved by the Planner, Writer, Reviewer, and final message.

The final JSON included both required fields:

{
  "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
  "status": "approved",
  "answer": "Before applying for a rental listing, a renter should verify the property's exact address and location, confirm the rental price and any additional fees, check the availability date and lease term length, ensure the landlord or agent is legitimate with verified contact information, and review the required documents and application process."
}

```
carol@Carols-MacBook-Air data260-2974-hw7 % PYTHONPATH=code .venv/bin/python code/send_question.py
[CLIENT] SENDING topic=s2974_inbox
{
  "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
  "question": "What information should a renter verify before applying for a rental listing?"
}
[CLIENT] SENT partition=0 offset=1
[CLIENT] RECEIVED FINAL topic=s2974_final partition=0 offset=1
[CLIENT] FINAL JSON
{
  "answer": "What information should a renter verify before applying for a rental listing?\n\nBefore applying for a rental listing, a renter should verify the property's exac
t address and location, confirm the rental price and any additional fees, check the availability date and lease term length, ensure the landlord or agent is legitimate with v
erified contact information, and review the required documents and application process.",
  "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
  "status": "approved"
}
```

Figure 7. Client request and final approved JSON

The Planner consumed the original question, generated a plan, published it to s2974_tasks, and committed its input offset.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs --tail=60 planner
s2974-hw8-planner  | [PLANNER] READY input=s2974_inbox output=s2974_tasks model=qwen3:8b
s2974-hw8-planner  | [PLANNER] RECEIVED topic=s2974_inbox partition=0 offset=0 correlation_id=ac993c1a-82f9-4dd0-9a7e-2683b144193c
s2974-hw8-planner  | [PLANNER] INPUT {
s2974-hw8-planner  |   "correlation_id": "ac993c1a-82f9-4dd0-9a7e-2683b144193c",
s2974-hw8-planner  |   "question": "What information should a renter verify before applying for a rental listing?"
s2974-hw8-planner  | }
s2974-hw8-planner  | [PLANNER] OUTPUT {
s2974-hw8-planner  |   "correlation_id": "ac993c1a-82f9-4dd0-9a7e-2683b144193c",
s2974-hw8-planner  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check the
 availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and applic
ation process.",
s2974-hw8-planner  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-planner  |   "stage": "planned"
s2974-hw8-planner  | }
s2974-hw8-planner  | [PLANNER] SENT topic=s2974_tasks partition=0 offset=0 correlation_id=ac993c1a-82f9-4dd0-9a7e-2683b144193c
s2974-hw8-planner  | [PLANNER] COMMITTED input_offset=0
s2974-hw8-planner  | [PLANNER] RECEIVED topic=s2974_inbox partition=0 offset=1 correlation_id=1524d90b-c744-4303-995f-ed4d9c2d43de
s2974-hw8-planner  | [PLANNER] INPUT {
s2974-hw8-planner  |   "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
s2974-hw8-planner  |   "question": "What information should a renter verify before applying for a rental listing?"
s2974-hw8-planner  | }
s2974-hw8-planner  | [PLANNER] OUTPUT {
s2974-hw8-planner  |   "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
s2974-hw8-planner  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check the
 availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and applic
ation process.",
s2974-hw8-planner  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-planner  |   "stage": "planned"
s2974-hw8-planner  | }
s2974-hw8-planner  | [PLANNER] SENT topic=s2974_tasks partition=0 offset=1 correlation_id=1524d90b-c744-4303-995f-ed4d9c2d43de
s2974-hw8-planner  | [PLANNER] COMMITTED input_offset=1
```

Figure 8. Planner input, plan output, Kafka send, and commit

The Writer consumed the plan, generated a plain-language draft with local qwen3:8b, published it to s2974_drafts, and committed its input offset.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs --tail=60 writer
s2974-hw8-writer  | [WRITER] READY input=s2974_tasks output=s2974_drafts model=qwen3:8b
s2974-hw8-writer  | [WRITER] RECEIVED topic=s2974_tasks partition=0 offset=0 correlation_id=ac993c1a-82f9-4dd0-9a7e-2683b144193c
s2974-hw8-writer  | [WRITER] INPUT {
s2974-hw8-writer  |   "correlation_id": "ac993c1a-82f9-4dd0-9a7e-2683b144193c",
s2974-hw8-writer  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check the
availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and applica
tion process.",
s2974-hw8-writer  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-writer  |   "stage": "planned"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] OUTPUT {
s2974-hw8-writer  |   "correlation_id": "ac993c1a-82f9-4dd0-9a7e-2683b144193c",
s2974-hw8-writer  |   "draft": "Before applying for a rental listing, a renter should verify the property's exact address and location, confirm the rental price and any addit
ional fees, check the availability date and lease term length, ensure the landlord or agent is legitimate with verified contact information, and review the required documents
 and application process.",
s2974-hw8-writer  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check the
availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and applica
tion process.",
s2974-hw8-writer  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-writer  |   "stage": "drafted"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] SENT topic=s2974_drafts partition=0 offset=0 correlation_id=ac993c1a-82f9-4dd0-9a7e-2683b144193c
s2974-hw8-writer  | [WRITER] COMMITTED input_offset=0
s2974-hw8-writer  | [WRITER] RECEIVED topic=s2974_tasks partition=0 offset=1 correlation_id=1524d90b-c744-4303-995f-ed4d9c2d43de
s2974-hw8-writer  | [WRITER] INPUT {
s2974-hw8-writer  |   "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
s2974-hw8-writer  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check the
availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and applica
tion process.",
s2974-hw8-writer  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-writer  |   "stage": "planned"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] OUTPUT {
s2974-hw8-writer  |   "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
s2974-hw8-writer  |   "draft": "Before applying for a rental listing, a renter should verify the property's exact address and location, confirm the rental price and any addit
ional fees, check the availability date and lease term length, ensure the landlord or agent is legitimate with verified contact information, and review the required documents
 and application process.",
s2974-hw8-writer  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check the
availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and applica
tion process.",
s2974-hw8-writer  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-writer  |   "stage": "drafted"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] SENT topic=s2974_drafts partition=0 offset=1 correlation_id=1524d90b-c744-4303-995f-ed4d9c2d43de
s2974-hw8-writer  | [WRITER] COMMITTED input_offset=1
```

Figure 9. Writer input, draft output, Kafka send, and commit

The Reviewer consumed the draft, approved it, published the final JSON to s2974_final, and committed its input offset.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs --tail=60 reviewer
s2974-hw8-reviewer  | [REVIEWER] READY input=s2974_drafts output=s2974_final model=qwen3:8b
s2974-hw8-reviewer  | [REVIEWER] RECEIVED topic=s2974_drafts partition=0 offset=0 correlation_id=ac993c1a-82f9-4dd0-9a7e-2683b144193c
s2974-hw8-reviewer  | [REVIEWER] INPUT {
s2974-hw8-reviewer  |   "correlation_id": "ac993c1a-82f9-4dd0-9a7e-2683b144193c",
s2974-hw8-reviewer  |   "draft": "Before applying for a rental listing, a renter should verify the property's exact address and location, confirm the rental price and any add
itional fees, check the availability date and lease term length, ensure the landlord or agent is legitimate with verified contact information, and review the required documen
ts and application process.",
s2974-hw8-reviewer  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check th
e availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and appli
cation process.",
s2974-hw8-reviewer  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-reviewer  |   "stage": "drafted"
s2974-hw8-reviewer  | }
s2974-hw8-reviewer  | [REVIEWER] OUTPUT {
s2974-hw8-reviewer  |   "answer": "What information should a renter verify before applying for a rental listing?\n\nBefore applying for a rental listing, a renter should veri
fy the property's exact address and location, confirm the rental price and any additional fees, check the availability date and lease term length, ensure the landlord or agen
t is legitimate with verified contact information, and review the required documents and application process.",
s2974-hw8-reviewer  |   "correlation_id": "ac993c1a-82f9-4dd0-9a7e-2683b144193c",
s2974-hw8-reviewer  |   "status": "approved"
s2974-hw8-reviewer  | }
s2974-hw8-reviewer  | [REVIEWER] SENT topic=s2974_final partition=0 offset=0 correlation_id=ac993c1a-82f9-4dd0-9a7e-2683b144193c
s2974-hw8-reviewer  | [REVIEWER] COMMITTED input_offset=0
s2974-hw8-reviewer  | [REVIEWER] RECEIVED topic=s2974_drafts partition=0 offset=1 correlation_id=1524d90b-c744-4303-995f-ed4d9c2d43de
s2974-hw8-reviewer  | [REVIEWER] INPUT {
s2974-hw8-reviewer  |   "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
s2974-hw8-reviewer  |   "draft": "Before applying for a rental listing, a renter should verify the property's exact address and location, confirm the rental price and any add
itional fees, check the availability date and lease term length, ensure the landlord or agent is legitimate with verified contact information, and review the required documen
ts and application process.",
s2974-hw8-reviewer  |   "plan": "1. Confirm the property's exact address and location.  \n2. Verify the rental price, including any additional fees or charges.  \n3. Check th
e availability date and lease term length.  \n4. Ensure the landlord or agent is legitimate and contact information is verified.  \n5. Review the required documents and appli
cation process.",
s2974-hw8-reviewer  |   "question": "What information should a renter verify before applying for a rental listing?",
s2974-hw8-reviewer  |   "stage": "drafted"
s2974-hw8-reviewer  | }
s2974-hw8-reviewer  | [REVIEWER] OUTPUT {
s2974-hw8-reviewer  |   "answer": "What information should a renter verify before applying for a rental listing?\n\nBefore applying for a rental listing, a renter should veri
fy the property's exact address and location, confirm the rental price and any additional fees, check the availability date and lease term length, ensure the landlord or agen
t is legitimate with verified contact information, and review the required documents and application process.",
s2974-hw8-reviewer  |   "correlation_id": "1524d90b-c744-4303-995f-ed4d9c2d43de",
s2974-hw8-reviewer  |   "status": "approved"
s2974-hw8-reviewer  | }
s2974-hw8-reviewer  | [REVIEWER] SENT topic=s2974_final partition=0 offset=1 correlation_id=1524d90b-c744-4303-995f-ed4d9c2d43de
s2974-hw8-reviewer  | [REVIEWER] COMMITTED input_offset=1
```

Figure 10. Reviewer approval, final output, Kafka send, and commit

The required s2974_tasks topic was verified to contain two partitions.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml exec kafka \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic s2974_tasks
Topic: s2974_tasks      TopicId: Ike37n3TRv-BLkmyEiZg5A PartitionCount: 2       ReplicationFactor: 1    Configs:
        Topic: s2974_tasks      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: s2974_tasks      Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
```

Figure 11. Two-partition configuration for s2974_tasks

2.3 How Kafka Coordinated the Agents

Kafka provided a durable boundary between the agents. Each agent only needed to understand its input and output message schema; it did not need to know how another agent was implemented or whether that process was temporarily unavailable.

Kafka also allowed each stage to commit its input offset after publishing its output. This made recovery observable and allowed failed work to be redelivered. However, this design also required explicit handling for duplicate delivery, poison messages, partition ordering, and safe offset commits.

Part 3 – Failure Behavior

3.1 Failure Comparison Table

Failure scenario	Before fix or failure evidence	After fix or recovery evidence
Duplicate delivery	Replaying one correlation_id created two different entity IDs, producing an extra side effect.	An idempotency cache returned the original response and entity ID. The replay produced no additional entity.
Consumer killed mid-work	Writer was killed after receiving partition 1, offset 0 but before committing it. The container exited with code 137.	After restart, Writer received partition 1, offset 0 again, completed processing, committed the offset, and produced an approved result.
Poison message	The original processing path had no bounded exception/DLQ path. A ValueError: plan is required could prevent the task offset from advancing.	Writer attempted the message three times, moved it to s2974_dead_letter, committed the source offset, and processed the next valid message.
Two partitions/two consumers	Without an entity key, 20 related messages were split across partitions 0 and 1 and processed by two Writers. Global order was not preserved.	With property-6-262974 as the key, all 20 messages stayed in partition 0 and were processed in sequence 1–20.

3.2 Duplicate Delivery

Before the Fix

The same correlation ID was replayed: 2c665ed1-14ce-434a-be14-d74d32446369

The first and second deliveries created different entities:

first_entity_id = ddb65d89-2614-4044-8634-6025ea3cb8e3
second_entity_id = 690ac07b-c3f4-494d-95fa-3b69f4077744
extra_side_effect = true

```
carol@Carols-MacBook-Air data260-2974-hw7 % cat reports/hw08/raw/duplicate_before_fix.json
{
  "correlation_id": "2c665ed1-14ce-434a-be14-d74d32446369",
  "extra_side_effect": true,
  "first_entity_id": "ddb65d89-2614-4044-8634-6025ea3cb8e3",
  "first_response": {
    "correlation_id": "2c665ed1-14ce-434a-be14-d74d32446369",
    "entity_id": "ddb65d89-2614-4044-8634-6025ea3cb8e3",
    "message": "Rental listing created",
    "success": true
  },
  "label": "before_fix",
  "same_correlation_id": true,
  "same_entity_id": false,
  "scenario": "duplicate_delivery",
  "second_entity_id": "690ac07b-c3f4-494d-95fa-3b69f4077744",
  "second_response": {
    "correlation_id": "2c665ed1-14ce-434a-be14-d74d32446369",
    "entity_id": "690ac07b-c3f4-494d-95fa-3b69f4077744",
    "message": "Rental listing created",
    "success": true
  }
}
```

Figure 12. Duplicate delivery before idempotency

After the Fix

The Worker stores a fingerprint and response for each successfully processed correlation ID. An exact replay returns a copy of the stored response.

first_entity_id = 604ad416-edc4-425f-b49a-e6e534349094
second_entity_id = 604ad416-edc4-425f-b49a-e6e534349094
idempotent_replay = true
extra_side_effect = false

```
carol@Carols-MacBook-Air data260-2974-hw7 % PYTHONPATH=code .venv/bin/python code/hw8_duplicate_replay.py \
  --label after_fix \
  --output reports/hw08/raw/duplicate_after_fix.json
[DUPLICATE TEST] Sending original request
{
  "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
  "data": {
    "address": "2974 Replay Avenue, San Jose, CA",
    "availableUnits": 1,
    "description": "A deterministic listing used to test duplicate Kafka delivery.",
    "listingTitle": "Duplicate Replay Test Listing",
    "monthlyRent": 2297,
    "propertyType": "Apartment"
  },
  "operation": "CREATE_LISTING",
  "reply_to": "s2974_entity_responses"
}
[DUPLICATE TEST] First response
{
  "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
  "entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
  "message": "Rental listing created",
  "success": true
}
[DUPLICATE TEST] Replaying identical correlation_id
[DUPLICATE TEST] Second response
{
  "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
  "entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
  "idempotent_replay": true,
  "message": "Rental listing created",
  "success": true
}
[DUPLICATE TEST] Summary
{
  "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
  "extra_side_effect": false,
  "first_entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
  "first_response": {
    "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
    "entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
    "message": "Rental listing created",
    "success": true
  },
  "label": "after_fix",
  "same_correlation_id": true,
  "same_entity_id": true,
  "scenario": "duplicate_delivery",
  "second_entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
  "second_response": {
    "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
    "entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
    "idempotent_replay": true,
    "message": "Rental listing created",
    "success": true
  }
}
[DUPLICATE TEST] Wrote reports/hw08/raw/duplicate_after_fix.json
```

Figure 13. Duplicate delivery after idempotency

The Worker log also explicitly recorded the replay and confirmed that the entity count did not increase.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs --tail=100 worker
s2974-hw8-worker  | Consumer is ready and listening... topic=s2974_entity_requests
s2974-hw8-worker  | [WORKER] RECEIVED topic=s2974_entity_requests partition=0 offset=4
s2974-hw8-worker  | [WORKER] REQUEST {
s2974-hw8-worker  |   "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
s2974-hw8-worker  |   "data": {
s2974-hw8-worker  |     "address": "2974 Replay Avenue, San Jose, CA",
s2974-hw8-worker  |     "availableUnits": 1,
s2974-hw8-worker  |     "description": "A deterministic listing used to test duplicate Kafka delivery.",
s2974-hw8-worker  |     "listingTitle": "Duplicate Replay Test Listing",
s2974-hw8-worker  |     "monthlyRent": 2297,
s2974-hw8-worker  |     "propertyType": "Apartment"
s2974-hw8-worker  |   },
s2974-hw8-worker  |   "operation": "CREATE_LISTING",
s2974-hw8-worker  |   "reply_to": "s2974_entity_responses"
s2974-hw8-worker  | }
s2974-hw8-worker  | [WORKER] PROCESSED operation=CREATE_LISTING correlation_id=807ddbd6-e574-48dd-8395-872307fa2566
s2974-hw8-worker  | [WORKER] RESPONSE {
s2974-hw8-worker  |   "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
s2974-hw8-worker  |   "entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
s2974-hw8-worker  |   "message": "Rental listing created",
s2974-hw8-worker  |   "success": true
s2974-hw8-worker  | }
s2974-hw8-worker  | [WORKER] SENT topic=s2974_entity_responses partition=0 offset=4
s2974-hw8-worker  | [WORKER] COMMITTED request_offset=4
s2974-hw8-worker  | [WORKER] RECEIVED topic=s2974_entity_requests partition=0 offset=5
s2974-hw8-worker  | [WORKER] REQUEST {
s2974-hw8-worker  |   "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
s2974-hw8-worker  |   "data": {
s2974-hw8-worker  |     "address": "2974 Replay Avenue, San Jose, CA",
s2974-hw8-worker  |     "availableUnits": 1,
s2974-hw8-worker  |     "description": "A deterministic listing used to test duplicate Kafka delivery.",
s2974-hw8-worker  |     "listingTitle": "Duplicate Replay Test Listing",
s2974-hw8-worker  |     "monthlyRent": 2297,
s2974-hw8-worker  |     "propertyType": "Apartment"
s2974-hw8-worker  |   },
s2974-hw8-worker  |   "operation": "CREATE_LISTING",
s2974-hw8-worker  |   "reply_to": "s2974_entity_responses"
s2974-hw8-worker  | }
s2974-hw8-worker  | [WORKER] IDEMPOTENT REPLAY correlation_id=807ddbd6-e574-48dd-8395-872307fa2566 entity_count=1
s2974-hw8-worker  | [WORKER] PROCESSED operation=CREATE_LISTING correlation_id=807ddbd6-e574-48dd-8395-872307fa2566
s2974-hw8-worker  | [WORKER] RESPONSE {
s2974-hw8-worker  |   "correlation_id": "807ddbd6-e574-48dd-8395-872307fa2566",
s2974-hw8-worker  |   "entity_id": "604ad416-edc4-425f-b49a-e6e534349094",
s2974-hw8-worker  |   "idempotent_replay": true,
s2974-hw8-worker  |   "message": "Rental listing created",
s2974-hw8-worker  |   "success": true
s2974-hw8-worker  | }
s2974-hw8-worker  | [WORKER] SENT topic=s2974_entity_responses partition=0 offset=5
s2974-hw8-worker  | [WORKER] COMMITTED request_offset=5
```

Figure 14. Worker idempotent-replay log

3.3 Consumer Killed Mid-Work

The test message used an intentional 60-second processing delay. Writer received:

topic=s2974_tasks
partition=1
offset=0

The container was killed with `SIGKILL` while the message was being processed. Docker reported: Exited (137)

Because the input offset had not been committed, Kafka retained the message as uncompleted work.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs --tail=30 writer
s2974-hw8-writer  | [WRITER] READY input=s2974_tasks output=s2974_drafts model=qwen3:8b
s2974-hw8-writer  | [WRITER] RECEIVED topic=s2974_tasks partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54
s2974-hw8-writer  | [WRITER] INPUT {
s2974-hw8-writer  |   "correlation_id": "4436d794-d1ce-493c-89ac-1a3394cb7e54",
s2974-hw8-writer  |   "failure_scenario": "consumer_death",
s2974-hw8-writer  |   "plan": "Explain how to verify the listing, costs, lease terms, and landlord.",
s2974-hw8-writer  |   "processing_delay_seconds": 60.0,
s2974-hw8-writer  |   "question": "What should a renter verify before signing a lease?",
s2974-hw8-writer  |   "stage": "planned"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] PROCESSING STARTED partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54 delay_seconds=60
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml kill -s SIGKILL writer
[+] kill 1/1
 ✔ Container s2974-hw8-writer Killed
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml ps -a writer
NAME                IMAGE                     COMMAND              SERVICE   CREATED              STATUS                          PORTS
s2974-hw8-writer    data260-2974-hw7-writer   "python writer.py"   writer    About a minute ago   Exited (137) 7 seconds ago
```

Figure 15. Writer killed after processing started and before commit

After Writer restarted, it received the same partition and offset: partition=1 offset=0

It then completed the delay, produced the draft, and committed the input offset.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs --tail=80 -f writer
s2974-hw8-writer  | [WRITER] READY input=s2974_tasks output=s2974_drafts model=qwen3:8b
s2974-hw8-writer  | [WRITER] RECEIVED topic=s2974_tasks partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54
s2974-hw8-writer  | [WRITER] INPUT {
s2974-hw8-writer  |   "correlation_id": "4436d794-d1ce-493c-89ac-1a3394cb7e54",
s2974-hw8-writer  |   "failure_scenario": "consumer_death",
s2974-hw8-writer  |   "plan": "Explain how to verify the listing, costs, lease terms, and landlord.",
s2974-hw8-writer  |   "processing_delay_seconds": 60.0,
s2974-hw8-writer  |   "question": "What should a renter verify before signing a lease?",
s2974-hw8-writer  |   "stage": "planned"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] PROCESSING STARTED partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54 delay_seconds=60
s2974-hw8-writer  | [WRITER] READY input=s2974_tasks output=s2974_drafts model=qwen3:8b
s2974-hw8-writer  | [WRITER] RECEIVED topic=s2974_tasks partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54
s2974-hw8-writer  | [WRITER] INPUT {
s2974-hw8-writer  |   "correlation_id": "4436d794-d1ce-493c-89ac-1a3394cb7e54",
s2974-hw8-writer  |   "failure_scenario": "consumer_death",
s2974-hw8-writer  |   "plan": "Explain how to verify the listing, costs, lease terms, and landlord.",
s2974-hw8-writer  |   "processing_delay_seconds": 60.0,
s2974-hw8-writer  |   "question": "What should a renter verify before signing a lease?",
s2974-hw8-writer  |   "stage": "planned"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] PROCESSING STARTED partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54 delay_seconds=60
s2974-hw8-writer  | [WRITER] DELAY COMPLETE partition=1 offset=0 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54
s2974-hw8-writer  | [WRITER] OUTPUT {
s2974-hw8-writer  |   "correlation_id": "4436d794-d1ce-493c-89ac-1a3394cb7e54",
s2974-hw8-writer  |   "draft": "Before signing a lease, a renter should verify the listing, costs, lease terms, and landlord. First, confirm that the rental listing is legiti
mate by checking the property's address and ensuring it matches the listing. Next, review all costs, including rent, utilities, and any additional fees, to avoid unexpected c
harges. Carefully read the lease terms to understand the duration, rules, and responsibilities of both parties. Finally, verify the landlord's identity and contact informatio
n to ensure you're renting from a legitimate owner.",
s2974-hw8-writer  |   "plan": "Explain how to verify the listing, costs, lease terms, and landlord.",
s2974-hw8-writer  |   "question": "What should a renter verify before signing a lease?",
s2974-hw8-writer  |   "stage": "drafted"
s2974-hw8-writer  | }
s2974-hw8-writer  | [WRITER] SENT topic=s2974_drafts partition=0 offset=2 correlation_id=4436d794-d1ce-493c-89ac-1a3394cb7e54
s2974-hw8-writer  | [WRITER] COMMITTED input_offset=0
^C
```

Figure 16. Same offset redelivered after Writer restart

The final machine-readable result recorded: completed_after_restart = true status = approved

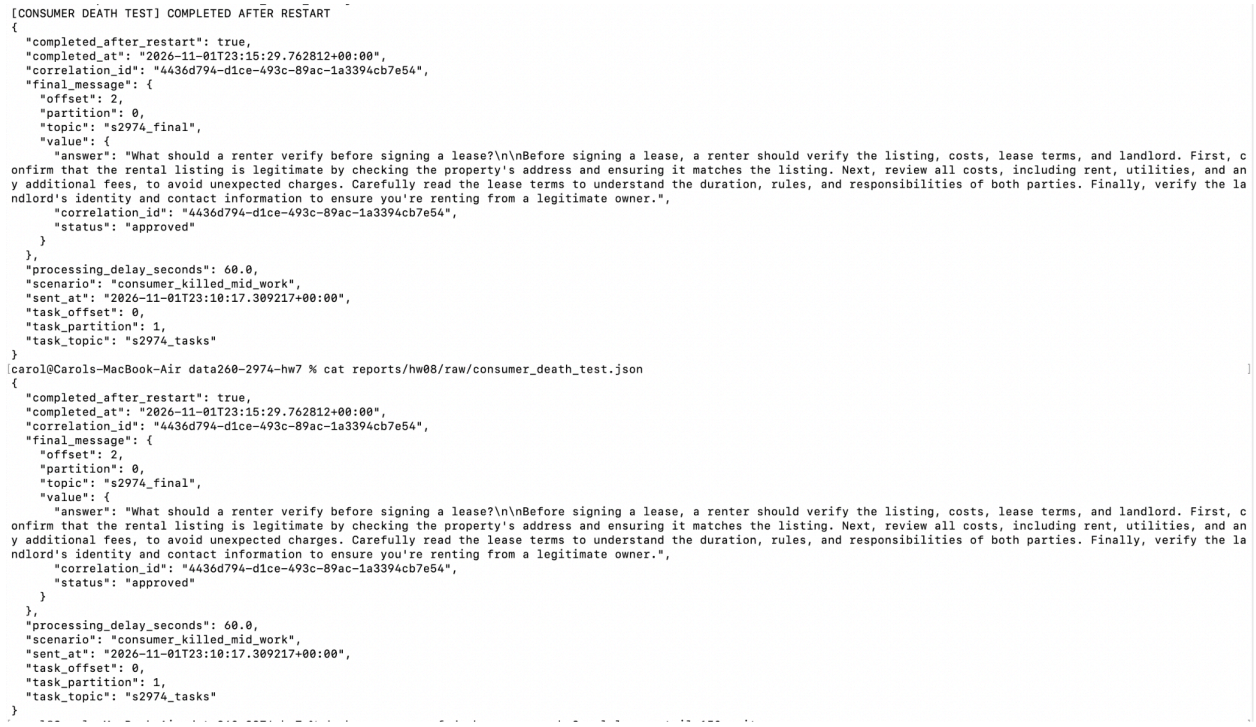


Figure 17. Approved final result after consumer restart

This demonstrates at-least-once delivery: an uncommitted message can be delivered again after a consumer failure.

3.4 Poison Message, Bounded Retries, and DLQ

The poison task intentionally omitted the required `plan` field. Writer reported: ValueError: plan is required

The retry policy allowed a maximum of three attempts: ATTEMPT 1/3 ATTEMPT 2/3 ATTEMPT 3/3

After the third failure, Writer published a failure record to: s2974_dead_letter

It committed the original task offset only after the dead-letter record was published.

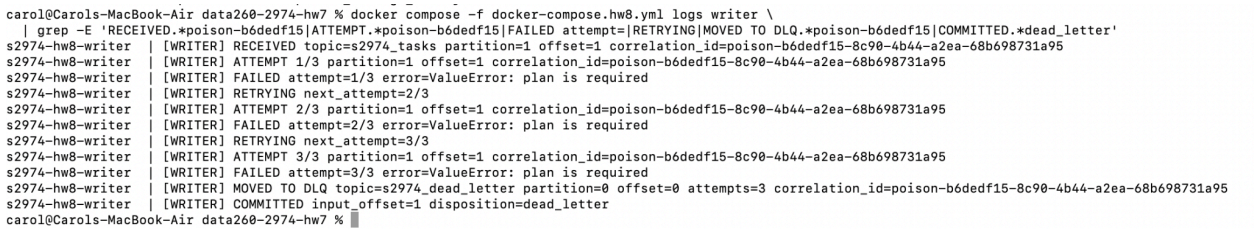


Figure 18. Three attempts followed by movement to the DLQ

The dead-letter record preserved the diagnostic information:

attempts = 3
failed_agent = WRITER
error_type = ValueError
error_message = plan is required
source_topic = s2974_tasks
source_partition = 1
source_offset = 1

```
carol@Carols-MacBook-Air data260-2974-hw7 % .venv/bin/python -c 'import json; p=json.load(open("reports/hw08/raw/poison_message_test.json")); print(json.dumps(p["dead_letter_
result"], indent=2, sort_keys=True))'
{
  "offset": 0,
  "partition": 0,
  "topic": "s2974_dead_letter",
  "value": {
    "attempts": 3,
    "correlation_id": "poison-b6dedf15-8c90-4b44-a2ea-68b698731a95",
    "error_message": "plan is required",
    "error_type": "ValueError",
    "failed_agent": "WRITER",
    "original_message": {
      "correlation_id": "poison-b6dedf15-8c90-4b44-a2ea-68b698731a95",
      "failure_scenario": "poison_message",
      "question": "This poison task intentionally has no plan field.",
      "stage": "planned"
    },
    "raw_value": "{\"correlation_id\": \"poison-b6dedf15-8c90-4b44-a2ea-68b698731a95\", \"stage\": \"planned\", \"question\": \"This poison task intentionally has no plan fie
ld.\", \"failure_scenario\": \"poison_message\"}",
    "source_offset": 1,
    "source_partition": 1,
    "source_topic": "s2974_tasks"
  }
}
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 19. Dead-letter record and original failure context

A valid message immediately followed the poison task in the same partition: poison task offset = 1 later valid task offset = 2

The later task reached s2974_final with: status = approved later_message_completed = true

```
carol@Carols-MacBook-Air data260-2974-hw7 % .venv/bin/python -c 'import json; p=json.load(open("reports/hw08/raw/poison_message_test.json")); print(json.dumps({"poison_task":
p["poison_task"],"later_valid_task":p["later_valid_task"],"later_final_result":p["later_final_result"],"later_message_completed":p["later_message_completed"],"same_task_parti
tion":p["same_task_partition"]}, indent=2, sort_keys=True))'
{
  "later_final_result": {
    "offset": 3,
    "partition": 0,
    "topic": "s2974_final",
    "value": {
      "answer": "Before paying a deposit, a renter should verify the property details, confirm the identity of the landlord, review the written lease agreement, understand al
l costs involved, and ensure they are using a secure payment method.",
      "correlation_id": "valid-b6dedf15-8c90-4b44-a2ea-68b698731a95",
      "status": "approved"
    }
  },
  "later_message_completed": true,
  "later_valid_task": {
    "correlation_id": "valid-b6dedf15-8c90-4b44-a2ea-68b698731a95",
    "offset": 2,
    "partition": 1,
    "topic": "s2974_tasks"
  },
  "poison_task": {
    "correlation_id": "poison-b6dedf15-8c90-4b44-a2ea-68b698731a95",
    "offset": 1,
    "partition": 1,
    "topic": "s2974_tasks"
  },
  "same_task_partition": true
}
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 20. Later valid message completes after poison task

Bounded retries prevent an invalid message from being retried forever, while the DLQ preserves the failure for later inspection.

3.5 Ordering with Two Partitions and Two Writers

Both ordering runs used:

VERIFY_SEED = 262974
message count = 20
related entity = property-6-262974

Run 1: No Message Key

Without a key, the related messages were distributed across partitions 0 and 1. Both Writer instances processed messages. Kafka preserved order inside each partition, but the combined global completion sequence differed from the original send sequence.

partitions used = [0, 1]
Writer instances used = 2
global_order_preserved = false
per_partition_order_preserved = true

```
carol@Carols-MacBook-Air data260-2974-hw7 % .venv/bin/python - <<'PY'
import json

p = json.load(open("reports/hw08/raw/ordering_test.json"))
x = p["no_key"]

print("[ORDERING BEFORE FIX: NO KEY]")
print("verify_seed:", p["verify_seed"])
print("message_count:", x["message_count"])
print("keys:", x["keys"])
print("partitions_used:", x["partitions_used"])
print("writer_instances_used:", x["writer_instances_used"])
print("sent_sequence:", x["sent_sequence"])
print("processed_sequence:", x["processed_sequence"])
print("global_order_preserved:", x["global_order_preserved"])
print("per_partition_order_preserved:", x["per_partition_order_preserved"])
PY
[ORDERING BEFORE FIX: NO KEY]
verify_seed: 262974
message_count: 20
keys: [None]
partitions_used: [0, 1]
writer_instances_used: ['34b2652a71fc', '93c3796c0972']
sent_sequence: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
processed_sequence: [4, 1, 7, 2, 10, 11, 3, 5, 6, 8, 9, 12, 13, 14, 15, 16, 17, 18, 19, 20]
global_order_preserved: False
per_partition_order_preserved: True
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 21. No-key run split across partitions with global reordering

Run 2: Entity-Based Message Key

The second run used the related entity ID as the Kafka key: key = property-6-262974

All 20 messages were mapped to partition 0. One Writer processed the entity's messages in their original order:

partitions used = [0]
processed sequence = [1, 2, 3, ..., 20]
global_order_preserved = true

```
carol@Carols-MacBook-Air data260-2974-hw7 % .venv/bin/python - <<'PY'
import json

p = json.load(open("reports/hw08/raw/ordering_test.json"))
x = p["no_key"]

print("[ORDERING BEFORE FIX: NO KEY]")
print("verify_seed:", p["verify_seed"])
print("message_count:", x["message_count"])
print("keys:", x["keys"])
print("partitions_used:", x["partitions_used"])
print("writer_instances_used:", x["writer_instances_used"])
print("sent_sequence:", x["sent_sequence"])
print("processed_sequence:", x["processed_sequence"])
print("global_order_preserved:", x["global_order_preserved"])
print("per_partition_order_preserved:", x["per_partition_order_preserved"])
PY
[ORDERING BEFORE FIX: NO KEY]
verify_seed: 262974
message_count: 20
keys: [None]
partitions_used: [0, 1]
writer_instances_used: ['34b2652a71fc', '93c3796c0972']
sent_sequence: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
processed_sequence: [4, 1, 7, 2, 10, 11, 3, 5, 6, 8, 9, 12, 13, 14, 15, 16, 17, 18, 19, 20]
global_order_preserved: False
per_partition_order_preserved: True
carol@Carols-MacBook-Air data260-2974-hw7 % .venv/bin/python - <<'PY'
import json

p = json.load(open("reports/hw08/raw/ordering_test.json"))
x = p["entity_key"]

print("[ORDERING AFTER FIX: ENTITY KEY]")
print("verify_seed:", p["verify_seed"])
print("message_count:", x["message_count"])
print("entity_id:", x["entity_id"])
print("keys:", x["keys"])
print("partitions_used:", x["partitions_used"])
print("writer_instances_used:", x["writer_instances_used"])
print("sent_sequence:", x["sent_sequence"])
print("processed_sequence:", x["processed_sequence"])
print("global_order_preserved:", x["global_order_preserved"])
PY
[ORDERING AFTER FIX: ENTITY KEY]
verify_seed: 262974
message_count: 20
entity_id: property-6-262974
keys: ['property-6-262974']
partitions_used: [0]
writer_instances_used: ['93c3796c0972']
sent_sequence: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
processed_sequence: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
global_order_preserved: True
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 22. Entity-key run stays in one partition and preserves order

The experiment used two live Writer instances in the same consumer group, and all six automated ordering checks passed.

```
carol@Carols-MacBook-Air data260-2974-hw7 % docker compose -f docker-compose.hw8.yml logs ordering-writer \
  | grep 'ORDERING-WRITER.*READY'
ordering-writer-1  | [ORDERING-WRITER] READY instance=34b2652a71fc group=s2974_writer input=s2974_tasks output=s2974_ordering_results
ordering-writer-2  | [ORDERING-WRITER] READY instance=93c3796c0972 group=s2974_writer input=s2974_tasks output=s2974_ordering_results
carol@Carols-MacBook-Air data260-2974-hw7 % .venv/bin/python - <<'PY'
import json

p = json.load(open("reports/hw08/raw/ordering_test.json"))

print("[ORDERING TEST FINAL CHECKS]")
for name, passed in p["checks"].items():
    print(f"{name}: {passed}")
print("pass:", p["pass"])
PY
[ORDERING TEST FINAL CHECKS]
entity_key_preserved_order: True
entity_key_used_one_partition: True
no_key_global_order_changed: True
no_key_used_two_partitions: True
no_key_used_two_writers: True
per_partition_order_preserved: True
pass: True
carol@Carols-MacBook-Air data260-2974-hw7 %
```

Figure 23. Two Writer instances and final ordering checks

3.6 HW2 In-Process Graph vs. Kafka

Benefits Gained

1. Independent scaling

In HW2, every graph step ran inside one process. In the Kafka design, Planner, Writer, and Reviewer are independent consumers. A slow stage can be replicated without duplicating the entire application.

2. Decoupled failure and recovery

In the in-process graph, one failed function call directly interrupted the workflow. Kafka retains uncommitted messages, allowing a restarted consumer to resume or repeat unfinished work.

New Responsibilities

1. Duplicate delivery and idempotency

At-least-once delivery can repeat messages. Side-effecting operations therefore require stable correlation IDs, request fingerprints, and idempotent result handling.

2. Ordering, retry, and offset management

The application must select appropriate partition keys, understand that global ordering does not exist across partitions, bound retry attempts, maintain a dead-letter topic, and commit offsets only after successful processing or durable DLQ handling.

Final Verification

The final command was: make verify-hw08

The verifier:

- started or confirmed Kafka, kafka-init, Worker, and FastAPI;
- completed a live CREATE_LISTING → GET_LISTING round trip;
- ran six deterministic offline test suites containing 27 test cases;
- validated all five machine-readable Part 3 JSON files;
- confirmed all 23 labeled screenshots;
- checked the required documentation;
- confirmed that required evidence was tracked by Git;
- performed a count-only secret-pattern scan.

The final result was: FINAL_STATUS=PASS

Conclusion

The completed system demonstrates both the advantages and operational costs of Kafka-based coordination. Kafka decoupled the request/response service and the three agents, preserved unfinished work across consumer failure, and enabled independent consumers. The failure experiments also showed why production message systems require idempotency, bounded retries, dead-letter handling, careful offset commits, and entity-based partition keys.

All required live experiments, offline tests, machine-readable evidence files, and final verification checks passed.

Generative AI Use Disclosure

I used ChatGPT to clarify assignment requirements, organize the work into smaller stages, review code, and help interpret Kafka and Docker errors. I personally ran the services and experiments locally, inspected the real output, captured the screenshots, confirmed the results, and performed the Git operations.

One suggested docker events command did not return historical consumer-death events on my machine. I identified the problem because the output was empty. I independently verified the experiment using the existing Writer logs, Docker container status, the repeated Kafka partition and offset, and the final JSON result.

I replaced the unusable event evidence with the recorded PROCESSING STARTED line, the Exited (137) status, the repeated partition 1 / offset 0 after restart, and completed_after_restart=true. Together, these outputs demonstrate that the message was uncommitted when Writer died and was successfully redelivered after restart.